

Reporte Técnico de Actividades Práctico-Experimentales Nro. 001

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	José Riofrio, Francisco Jaramillo, Juan Alverca
Asignatura	Desarrollo Basado en Plataformas
Ciclo	5 A
Unidad	1
Resultado de aprendizaje de la unidad	
Práctica Nro.	001
Título de la Práctica	Implementar un servicio REST con Node.js
Nombre del Docente	Edison Leonardo Coronel Romero
Fecha	Viernes 3 de octubre
Horario	07h30 – 10h30
Lugar	Laboratorio Computación aplicada Laboratorio Desarrollo de Software Laboratorio de redes y Sistemas Operativos Laboratorio Virtual EVA Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

- Integrar la aplicación móvil (nativa o híbrida) con la API REST desarrollada en la Unidad 2 del proyecto.
- Validar la correcta comunicación cliente-servidor, verificando respuestas, códigos de estado y flujo de datos.
- Implementar y comprobar el manejo de errores en la aplicación móvil.
- Documentar técnica y visualmente el proceso de integración dentro del repositorio del proyecto

3. Materiales, Reactivos, Equipos y Herramientas

- Computador con acceso a Internet.
- Navegador web actualizado.
- Editor de código (VS Code o similar).
- Herramienta de prototipado Stitch (o entorno definido por la asignatura).
- Repositorio del proyecto en GitHub / GitLab.
- Lineamientos de UI/UX del proyecto (si ya existen).

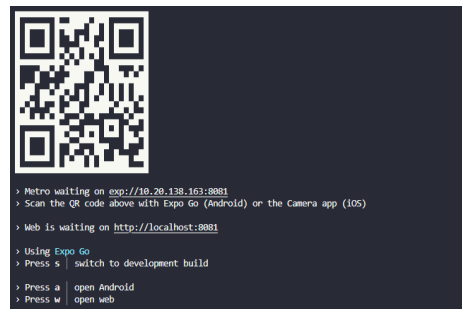
4. Procedimiento / Metodología Ejecutada

1. Revisión rápida de la arquitectura del proyecto.
2. Identificación de los endpoints REST que serán consumidos por la app móvil (por ejemplo: login, listado, registro).
3. Recordatorio de códigos de estado HTTP y tipos de errores comunes.

5. Resultados

Paso 1: Configuración de conexión

- Definir la URL base del backend (local, IP o dominio).
- Configurar variables de entorno o constantes para el consumo de la API.



```
AccessibilityContext.jsx  config.js  FIX_REGISTRO_AUTHSERVICE.md  INICIAR_SISTEMA_COMPLETO.md  ...
Truekealo > mobile > src > constants > config.js > ...

1
2
3 import { Platform } from 'react-native';
4
5 // Configuración por plataforma
6 const getApiUrl = () => {
7   // IMPORTANTE: Si estás usando Expo Go en dispositivo físico,
8   // SIEMPRE usa la IP de tu red local (no 10.0.2.2)
9
10  // Para dispositivo físico (Expo Go)
11  return 'http://10.20.138.163:8001/api/v1';
12
13  // Descomentar esto solo si usas Android Emulator:
14  // if (Platform.OS === 'android') {
15  //   return 'http://10.0.2.2:8001/api/v1';
16  // }
17  // return 'http://192.168.0.3:8001/api/v1';
18 };
19
20 export const API_URL = getApiUrl();
21
```

- La aplicación se comunica con el backend a través de una URL base configurada como `http://10.20.138.163:8001/api/v1`, para esta fase del proyecto se utiliza el protocolo HTTP, ya que el sistema se encuentra en un entorno de desarrollo local.
- El host corresponde a la dirección IP 10.20.138.163, la cual pertenece a la red local donde se ejecuta el servidor, el servicio backend está expuesto mediante el puerto 8001, utilizado por la aplicación desarrollada con FastAPI, finalmente, se emplea el prefijo `/api/v1` para el versionado de la API, lo que permite organizar mejor los endpoints y facilita futuras actualizaciones del sistema.

Paso 2: Consumo de la API REST

- Implementar al menos una funcionalidad completa, por ejemplo: autenticación,
- listado de datos,
- registro o envío de información.
- Realizar la petición HTTP desde la app móvil y procesar la respuesta.

```
(NOBRIDGE) LOG Login exitoso, usuario: francisco.2012@gmail.com
(NOBRIDGE) LOG Token obtenido: eyJhbGciOiJIUzI1NiIs...
(NOBRIDGE) LOG API Request: {"headers": {"Authorization": "****", "Content-Type": "application/json"}, "method": "GET", "url": "http://10.20.138.163:8001/api/v1/propuestas/recibidas"}
(NOBRIDGE) LOG API Response: {"ok": true, "status": 200, "url": "http://10.20.138.163:8001/api/v1/propuestas/recibidas"}
(NOBRIDGE) LOG Token obtenido: eyJhbGciOiJIUzI1NiIs...
(NOBRIDGE) LOG API Request: {"headers": {"Authorization": "****", "Content-Type": "application/json"}, "method": "GET", "url": "http://10.20.138.163:8001/api/v1/propuestas/enviadas"}
(NOBRIDGE) LOG API Response: {"ok": true, "status": 200, "url": "http://10.20.138.163:8001/api/v1/propuestas/enviadas"}
(NOBRIDGE) LOG Token obtenido: eyJhbGciOiJIUzI1NiIs...
(NOBRIDGE) LOG API Request: {"headers": {"Authorization": "****", "Content-Type": "application/json"}, "method": "GET", "url": "http://10.20.138.163:8001/api/v1/mensajes/conversaciones"}
(NOBRIDGE) LOG API Response: {"ok": true, "status": 200, "url": "http://10.20.138.163:8001/api/v1/mensajes/conversaciones"}
(NOBRIDGE) LOG Token obtenido: eyJhbGciOiJIUzI1NiIs...
(NOBRIDGE) LOG API Request: {"headers": {"Authorization": "****", "Content-Type": "application/json"}, "method": "GET", "url": "http://10.20.138.163:8001/api/v1/articulos/mis-articulos"}
(NOBRIDGE) LOG API Response: {"ok": true, "status": 200, "url": "http://10.20.138.163:8001/api/v1/articulos/mis-articulos"}
(NOBRIDGE) LOG Token obtenido: eyJhbGciOiJIUzI1NiIs...
(NOBRIDGE) LOG API Request: {"headers": {"Authorization": "****", "Content-Type": "application/json"}, "method": "GET", "url": "http://10.20.138.163:8001/api/v1/articulos"}
(NOBRIDGE) LOG API Response: {"ok": true, "status": 200, "url": "http://10.20.138.163:8001/api/v1/articulos"}
(NOBRIDGE) LOG Token obtenido: eyJhbGciOiJIUzI1NiIs...
(NOBRIDGE) LOG API Request: {"headers": {"Authorization": "****", "Content-Type": "application/json"}, "method": "GET", "url": "http://10.20.138.163:8001/api/v1/articulos/22"}
(NOBRIDGE) LOG API Response: {"ok": true, "status": 200, "url": "http://10.20.138.163:8001/api/v1/articulos/22"}
(NOBRIDGE) LOG Token obtenido: eyJhbGciOiJIUzI1NiIs...
```



Durante el consumo de la API REST se implementó correctamente el proceso de autenticación, evidenciado por el inicio de sesión exitoso y la obtención del token de acceso. A partir de este token, la aplicación móvil realizó peticiones HTTP de tipo GET a distintos endpoints del backend, permitiendo el listado de información como propuestas recibidas y enviadas, conversaciones, artículos propios y artículos generales. En todos los casos, las respuestas del servidor fueron procesadas de manera adecuada, obteniendo códigos de estado 200, lo que confirma la correcta comunicación entre la app móvil y el backend.

Paso 3: Manejo de errores

- Implementar manejo de:
- errores de red,
- respuestas 4xx y 5xx,
- datos inválidos o vacíos.
- Mostrar mensajes adecuados al usuario (alertas, mensajes visuales).

```
// En el método request() - catch block
catch (error) {
  console.error(`❌ Error en ${options.method || 'GET'} ${endpoint}:`, error.message)

  // Detectar errores de red
  if (error.message === 'Network request failed' || error.message.includes('fetch'))
    console.error('💡 Verifica que:');
    console.error('  - El backend esté corriendo en http://10.20.138.163:8001');
    console.error('  - Para Android Emulator, usa 10.0.2.2 (edita config.js)');
    console.error('  - Para dispositivo físico, usa 10.20.138.163 (actual)');
  }
  throw error;
}
```

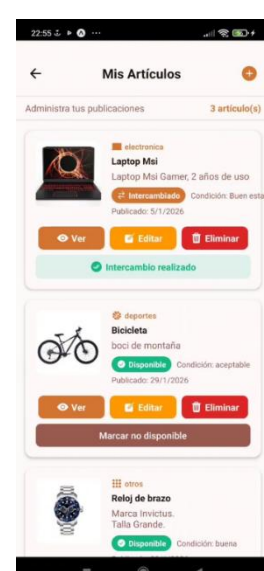
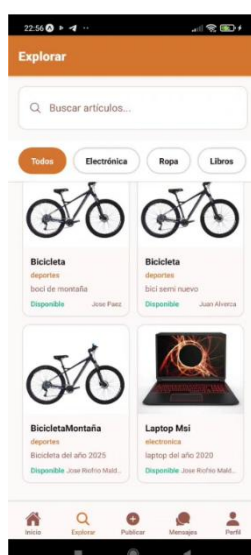
```
(NOBRIDGE) ERROR ❌ Error en GET /auth/me: Network request failed
(NOBRIDGE) ERROR 💡 Verifica que:
(NOBRIDGE) ERROR   - El backend esté corriendo en http://192.168.0.3:8001
(NOBRIDGE) ERROR   - Para Android Emulator, usa 10.0.2.2 (edita config.js)
(NOBRIDGE) ERROR   - Para dispositivo físico, usa 192.168.0.3 (actual)
```

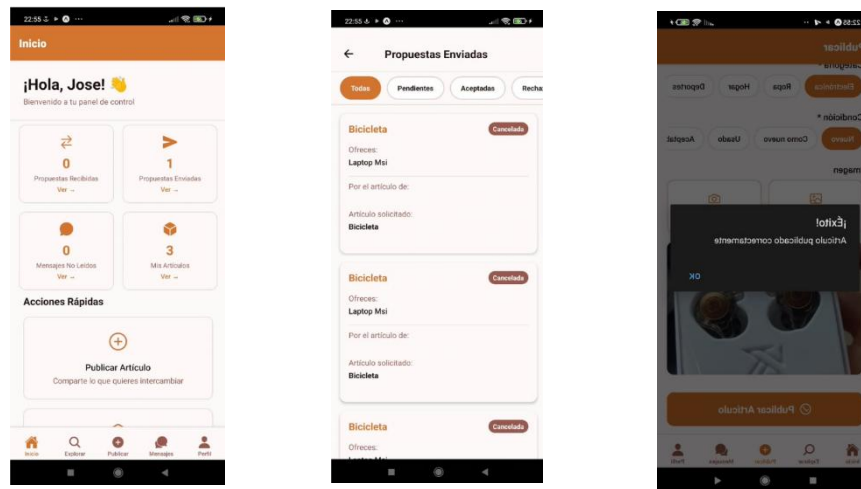
Para el consumo de la API REST, la aplicación móvil implementa un manejo básico de errores durante las peticiones HTTP. En caso de fallos de red o problemas de conexión, el sistema identifica el error y muestra mensajes de diagnóstico que permiten verificar si el backend se encuentra en ejecución y si la dirección IP configurada es la correcta según el entorno (emulador o dispositivo físico). Este manejo asegura una mejor detección de problemas de comunicación y facilita la validación de la conexión entre la aplicación móvil y el backend desarrollado con FastAPI.

Paso 4: Pruebas de integración

Probar la funcionalidad en emulador o dispositivo físico.

- Verificar:
- recepción correcta de datos,
- tiempos de respuesta,
- comportamiento ante errores simulados.





La funcionalidad de la aplicación fue probada tanto en emulador como en dispositivo físico, verificando la correcta recepción de datos desde el backend en las diferentes vistas del sistema. Durante las pruebas se comprobó que los tiempos de respuesta son adecuados, permitiendo una navegación fluida entre secciones como inicio, propuestas enviadas y publicación de artículos. Además, se evaluó el comportamiento de la aplicación ante errores simulados, validando que se muestren mensajes adecuados al usuario y que la interfaz responda de manera controlada sin afectar la experiencia de uso.

Paso 5: Documentación

En el repositorio del proyecto, documentar en un archivo (por ejemplo docs/integracion_app_movil.md):

- Endpoints consumidos.
- Ejemplo de solicitud y respuesta.
- Capturas de pantalla de la app en ejecución.
- Evidencia del manejo de errores.

Paso 6: Control de versiones

- Realizar commit descriptivo del avance.
- Enviar los cambios al repositorio remoto del equipo.

Cierre

- Socialización breve de resultados entre equipos.
- Retroalimentación

6. Resultados

- Aplicación móvil conectada correctamente al backend del proyecto.
- Evidencia funcional de consumo de la API REST.
- Manejo básico de errores implementado y probado.
- Documentación técnica clara dentro del repositorio.

7. Preguntas de Control

- **¿Qué endpoint del backend fue consumido por la aplicación móvil y con qué propósito?**

La aplicación móvil consumió varios endpoints del backend bajo el prefijo /api/v1, entre ellos /propuestas/recibidas, /propuestas/enviadas, /mensajes/conversaciones, /articulos/mis-articulos, /articulos y /articulos/{id}. Estos endpoints permiten autenticar al usuario mediante token, listar propuestas, mostrar conversaciones, visualizar artículos propios y generales, y consultar información detallada de un artículo específico.

- **¿Qué códigos de estado HTTP se validaron durante la práctica?**

Durante la práctica se validaron respuestas HTTP con código de estado 200 (OK), lo que indica que las solicitudes fueron procesadas correctamente por el servidor y que la comunicación entre la aplicación móvil y el backend se realizó sin errores.

- **¿Cómo se gestionan los errores de red o respuestas fallidas en la app móvil?**

La aplicación móvil implementa un bloque de manejo de errores que detecta fallos de red, como errores de tipo *Network request failed*. En estos casos, se muestran mensajes de diagnóstico en consola que permiten verificar si el backend está en ejecución y si la IP configurada es correcta según el entorno utilizado (emulador o dispositivo físico), facilitando la identificación del problema de conexión.

- **¿Qué diferencias encontraste entre consumir la API desde web y desde móvil?**

Una de las principales diferencias es la configuración de red, ya que en la aplicación móvil es necesario ajustar la dirección IP del backend dependiendo de si se usa un emulador o un dispositivo físico. A diferencia de la web, donde se puede consumir la API directamente desde localhost, en móvil se requiere usar direcciones específicas como 10.2.2.2 o la IP local real del servidor para evitar errores de conexión.

- **¿Qué mejoras aplicarías para fortalecer la experiencia del usuario en caso de error?**

Como mejora, se podría implementar mensajes visuales más claros para el usuario final en lugar de solo mensajes en consola, por ejemplo alertas o notificaciones amigables que indiquen problemas de conexión. Además, sería recomendable agregar reintentos automáticos de solicitud, indicadores de carga y mensajes personalizados que orienten al usuario sobre qué acción tomar cuando ocurra un error.

8. Conclusiones

- La aplicación móvil logró consumir correctamente la API REST desarrollada con FastAPI, validando procesos clave como la autenticación mediante token y la recuperación de datos desde distintos endpoints, lo que evidencia una comunicación estable y funcional entre el frontend móvil y el backend.
- Las pruebas realizadas en emulador y dispositivo físico permitieron confirmar que los tiempos de respuesta son adecuados y que la aplicación maneja de forma controlada los errores de red, garantizando una experiencia de uso coherente y sin fallos críticos durante la navegación.

9. Recomendaciones

- Se recomienda implementar mensajes de error visibles y amigables para el usuario final, acompañados de indicadores de carga, con el fin de mejorar la experiencia de usuario ante problemas de conexión o respuestas fallidas del servidor.
- Es aconsejable reforzar el manejo de errores mediante reintentos automáticos de las peticiones HTTP y optimizar la validación de la conectividad de red, especialmente al trabajar con diferentes entornos como emuladores y dispositivos físicos.

10. Recomendaciones

Link repositorio: <https://github.com/FrancisJaramilloC/SistemaTrueque.git>